CLASSIC DESIGN PATTERNS

COMMAND

Design Problem Resolution

Encapsulating a request as an object

A practical introduction with a Java example

AGENDA

01 Introduction

02 Real-world scenario

03 The design problem

04 What is Command?

05 Structure and participants

06 Original Java implementation

07 Command implementation and demo

08 Benefits and limitations

REAL-WORLD SCENARIO

The problem: a smart home remote

Imagine an app that controls lights, fans, and other devices.

User

Requests an action

Command Object

Represents the request

Light / Device

Performs the action

Without Command: UI code calls device methods directly.

With Command: each action is represented by a command object.

THE DESIGN PROBLEM

Common challenges

- User-interface code is tightly connected to device operations.
- Adding new actions can require changing existing code.
- Request-handling logic may be repeated.
- Undo, queueing, or logging actions can be difficult to add.

Design weakness

The client knows too much about how the receiver performs an action.

Resolution

Encapsulate the request in a Command object.

WHAT IS COMMAND?

Definition

The Command pattern turns a request into a stand-alone object containing the information needed to perform an action.

How it works

1 Client / Invoker

Requests an action

2 Command

Encapsulates the request

3 Receiver

Performs the action

The invoker triggers the command; the receiver carries out the real work.

STRUCTURE AND PARTICIPANTS

Invoker

The object that triggers the command. For example, a button or remote control.

Command interface

Defines a common method, usually called execute(), that concrete commands must implement.

Concrete Command

A specific implementation of the Command interface. It connects the requested action to the receiver.

Receiver

The object that knows how to perform the actual work.

ORIGINAL JAVA IMPLEMENTATION

Direct call: the client tells the light device what to do.

```
class Light {
    public void turnOn() {
        System.out.println("Light is ON");
    }
}

public class Main {
    public static void main(String[] args) {
        Light light = new Light();
        light.turnOn();
    }
}
```

Output

Light is ON

Issue

The client directly calls the device method, tying request handling to the receiver.

COMMAND IMPLEMENTATION AND DEMO

```
interface Command {
    void execute();
}

class Light {
    public void turnOn() {
        System.out.println("Light is ON");
    }
}

class TurnOnCommand implements Command {
    private Light light;
    TurnOnCommand(Light light) {
        this.light = light;
    }
    public void execute() {
        light.turnOn();
    }
}
```

Client demo

```
Light light = new Light();
Command cmd = new TurnOnCommand(light);
cmd.execute();
```

Output

Light is ON

The command object wraps the request and calls the receiver's method.

BENEFITS AND LIMITATIONS

Benefits

- Separates the invoker from the receiver.
- Makes it easier to add new commands.
- Can support queues, logging, and undo operations.
- Encapsulates each request as an object.

Limitations

- Creates additional classes and objects.
- May complicate a simple program.
- Each action may need its own command class.
- Undo requires extra state or reverse-action logic.

Use Command when separating, storing, queueing, or undoing requests is useful.

THANK YOU

Questions?